Structures de données pour l'unification en logique du premier ordre

Rapport en vue de la validation de l'UE Initiation à la recherche

Année universitaire 2025-2026

Etudiants :
Ayath ABOGOUNRIN
Guillaume POTEL
Matheo VIGNERES

Encadrants :
Julie CAILLER
Marie DUFLOT-KREMER
Engel LEFAUCHEUX

DÉCHARGE DE RESPONSABILITÉS

« L'Université de Lorraine n'entend donner ni approbation ni improbation aux opinions émises dans ce rapport, ces opinions devant être considérées comme propres à leur auteur. »

REMERCIEMENTS

A FAIRE

Sommaire

Introduction 1

I. Généralités 2

1. Logique des propositions 2
2. Logique du premier ordre 4
3. L'unification 7

II. Structures de données et algorithmes 8

1. Structures de données représentant un terme 8
   a. AST (*Abstract Syntax Tree*) 8
   b. Arbre avec étiquettes . . . . . ??

2. Structures de données représentant un ensemble de termes . . . . . ??
   a. Liste . . . . . ??
   b. Ensemble . . . . . ??
   c. Dictionnaire . . . . . ??

3. Algorithmes d'unification classiques . . . . . ??
   a. Martelli-Montanari . . . . . ??
   b. Robinson . . . . . ??

4. Discrimination Tree . . . . . ??

III. Résultats . . . . . ??

1. Protocole de benchmarks
2. Résultats
   a. Martelli-Montanari
   b. Robinson
   c. Discrimination Tree

Conclusion . . . . . ??

Introduction

En informatique, la représentation de l'information est un point clé dans la réalisation de divers projets. Ainsi le choix des **structures de données** représente une étape cruciale pour le bon déroulement de tout projet.

En logique du premier ordre, ce sont les **termes** qui servent de base afin de représenter des expressions symboliques construites à partir de symboles de fonction, de constantes et de variables. Représenter ces termes via la meilleure structure de donnée, la plus adéquate, représente donc un point majeur dans tout travaux portés à ce domaine.

L'**unification** est une opération essentielle dans ce cadre. Elle intervient notamment dans les moteurs d'inférence, les systèmes de réécriture de termes, et les prouveurs logiques. Les différents algorithmes connus d'unification sont parfois adaptés à certaines structures de données précises, il est donc important de comparer ces algorithmes et de diversifier ses choix pour parvenir aux résultats les plus efficaces.

Une structure de données est une représentation permettant d'organiser, traiter et stocker des données de manière efficace dans la mémoire d'un ordinateur. Elle permet un meilleur accès et une manipulation plus efficace des données afin de résoudre un problème spécifique.

Un terme est une expression qui consiste à nommer ou construire un objet en logique. Cela peut être une variable, une constante ou une fonction. Il est en quelque sorte le plus petit composant d'une formule logique.

L'unification consiste à déterminer s'il existe une substitution des variables (c'est-à-dire, une affectations des variables (X, Y , ...) vers des termes (a, b, f(a), ...)) permettant de rendre identiques deux termes.

Ainsi, dans le cadre de ce travail de recherche, nous nous demanderons quelles sont les structures de données les plus adaptées pour l'unification dans la logique du premier ordre. Le choix d'une structure impactant l'efficacité d'un algorithme, nous réfléchirons donc aussi aux meilleurs algorithmes d'unification à utiliser en fonction de la structure de donnée choisie.

Nous commencerons par un rappel sur les différentes notions abordées dans notre sujet. Ensuite nous présenterons plus en détails les différentes structures de données que nous avons utilisées, et nous finirons par la présentation des algorithmes d'unification connus sur lesquels nous avons travaillé.

I. Généralités

D'après nos connaissances, l'UE de L3 intitulée Logique des propositions - Logique du premier ordre*, de Jean Lieber et l'UE de M1 intitulée* Logiques et Modèles de Calcul*, de Didier Galmiche.*

Fil conducteur.

Tout au long de ce chapitre, nous allons construire progressivement les outils nécessaires pour répondre à une question très concrète : comment une machine peut-elle *démontrer* automatiquement que « Socrate est mortel » ? Ce raisonnement, connu sous le nom de *syllogisme*, peut sembler trivial au lecteur, mais sa mécanisation requiert précisément les notions que nous allons introduire : la logique des propositions, puis la logique du premier ordre, et enfin l'unification.

1. Logique des propositions

Logique formelle.

La *logique des propositions* (ou logique propositionnelle) constitue la brique fondamentale de la *logique formelle.*

Pour définir une logique formelle, on se donne un couple $(\mathcal{L}, \models)$ où

— $\mathcal{L}$ est un langage défini par une syntaxe. Une *formule* est un élément de $\mathcal{L}$. On appellera aussi *base de connaissances* un ensemble fini de formules.

— $\models$ est une *relation d'inférences* faisant le lien entre bases de connaissances et formules.

Remarque. *Si B est une base de connaissances et φ est une formule, l'expression $B \models \varphi$ se lit « B entraîne φ » et signifie que « Si l'on suppose que toutes les formules de B sont vraies, alors on peut en conclure que φ est vraie. ».*

Une introduction à la logique à destination d'étudiants de licence informatique - *Jean Lieber, p.4*

La logique des propositions permet donc de manipuler des formules construites à partir de *variables propositionnelles* et de *connecteurs logiques*, construits sur les bases de l'algèbre de Boole, et de leur attribuer une valeur de vérité. Pour décrire la logique propositionnelle, on peut utiliser la notation d'une logique formelle : $(\mathcal{LP}, \models)$.

Syntaxe de $(\mathcal{LP}, \models)$.

Une *formule propositionnelle* est une expression syntaxique construite selon la grammaire suivante :

$$\langle \text{formule} \rangle ::= \langle \text{variable propositionnelle} \rangle \mid \neg(\langle \text{formule} \rangle) \mid (\langle \text{formule} \rangle \ \langle \text{connecteur binaire} \rangle \ \langle \text{formule} \rangle)$$

$$\langle \text{connecteur binaire} \rangle ::= \wedge \mid \vee \mid \rightarrow \mid \leftrightarrow$$

Remarque. *Les connecteurs ont les lectures suivantes :*

$\neg$	$\wedge$	$\vee$	$\rightarrow$	$\leftrightarrow$
« non »	*« et »*	*« ou »*	*« implique »*	*« équivaut à »*

L'ensemble des variables se note $\mathcal{V}$. Il est généralement défini implicitement via les variables utilisées dans les formules.

Exemple :

Considérons les variables propositionnelles suivantes :

p	« Socrate est un homme »
q	« Tout homme est mortel »
r	« Socrate est mortel »

On peut alors former la formule :

$$(p \wedge q) \rightarrow r$$

qui se lit : « *Si* Socrate est un homme *et* tout homme est mortel, *alors* Socrate est mortel. »
Ici, $\mathcal{V} = \{p, q, r\}$.

Sémantique de $(\mathcal{LP}, \models)$.

Une *interprétation* $\mathcal{I}$ est une fonction qui associe à chaque variable propositionnelle une valeur de vérité dans $\mathbb{B} = \{\mathtt{V}, \mathtt{F}\}$. On étend naturellement $\mathcal{I}$ à toutes les formules en évaluant les connecteurs selon les règles booléennes usuelles. On dit que $\mathcal{I}$ est un *modèle* de φ (noté $\mathcal{I} \models \varphi$, se lit « $\mathcal{I}$ *satisfait* φ ») si la valeur de φ sous $\mathcal{I}$ est $\mathtt{V}$.

On peut étendre cette notion aux bases de connaissances : $\mathcal{I}$ satisfait B (noté $\mathcal{I} \models B$) si $\mathcal{I} \models \varphi$ pour tout $\varphi \in B$. Dans ce cas $\mathcal{I}$ est un modèle de B. On a aussi B entraîne φ (noté $B \models \varphi$) si tout modèle de B est un modèle de φ.

D'après la construction sémantique ci-dessus, on a :

1. $\varphi_1 \models \varphi_2$ si $\{\varphi_1\} \models \varphi_2$.
2. $\varphi_1 \equiv \varphi_2$ si $\varphi_1 \models \varphi_2$ et x$\varphi_2 \models \varphi_1$.
   $\varphi_1 \equiv \varphi_2$ signifie donc que I est un modèle de φ_1 ssi I est un modèle de φ_2.
3. φ est une *tautologie* (noté $\models \varphi$) si toute interprétation est un modèle de φ.
   On notera que $\models \varphi$ ssi $\varnothing \models \varphi$.
4. $\varphi \models_B \psi$ si $B \cup \{\varphi\} \models \psi$.

Remarque. *Une formule est dite* satisfiable *(cohérente, non contradictoire ou consistente) s'il existe au moins un modèle. On a le lien suivant : φ est une tautologie si et seulement si $\neg\varphi$ est* insatisfiable *(incohérente, contradictoire ou inconsistente).*

Une introduction à la logique à destination d'étudiants de licence informatique - *Jean Lieber, p.7*

Limites de $(\mathcal{LP}, \models)$.

La logique propositionnelle ne traite que d'un individu à la fois : elle est insuffisante pour exprimer des propriétés générales portant sur des ensembles d'objets ou des relations entre eux.

Revenons à notre syllogisme. On peut certes poser, comme dans l'exemple précédent :

$$\begin{array}{ll} p & \text{« Socrate est un homme »} \\ q & \text{« Tout homme est mortel »} \\ r & \text{« Socrate est mortel »} \end{array}$$

et former la tautologie $(p \wedge q) \to r$. Mais ce faisant, on *encode en dur* chaque fait : la proposition q est un bloc opaque ; on ne peut pas en extraire la règle générale « pour tout individu x, si x est un homme, alors x est mortel ». Si l'on voulait raisonner sur d'autres individus, e.g. Platon ou Aristote, il faudrait introduire autant de nouvelles variables propositionnelles, sans jamais capturer la généralité du raisonnement.

C'est précisément cette limite que lève la logique du premier ordre, présentée dans la section suivante, en introduisant des *variables* sur lesquelles on peut quantifier.

2. Logique du premier ordre

Pour pouvoir étendre le champs de couverture de la logique propositionnelle, on a défini la *logique du premier ordre* ou logique des prédicats (noté $(\mathcal{L}1\mathcal{O}, \models)$).

Cette logique permet la représentation de propriétés portant sur des ensembles d'objets ou des relations.

Syntaxe de $(\mathcal{L}1\mathcal{O}, \models)$.

Pour représenter des formules dans $(\mathcal{L}1\mathcal{O}, \models)$, on se donne un *alphabet* qui servira à les décrire. Cet alphabet est composé des éléments suivants :

— Ensemble de *variables* : $x, y, z, \ldots$

On définit également ce que l'on appelle une *signature*, qui rend notre représentation unique, composée de deux ensembles :

— Ensemble de symboles de *fonctions* : $f, g, h, \ldots$

— Ensemble de symboles de *prédicats* : $P, Q, R, \ldots$

On associe à ces fonctions et prédicats une *arité* (c.-à-d. le nombre d'arguments qu'ils prennent). Parmi les symboles de fonctions, ceux d'arité 0 sont appelés *constantes* :

— Ensemble de symboles de constantes : $a, b, c, \ldots$

> **Remarque.** *Le choix de ces différents symboles est purement arbitraire : on pourrait nommer les variables a, b ou c. Dans un souci d'harmonisation et de standardisation, on utilisera néanmoins les symboles les plus courants dans la littérature.*

Pour définir la structure d'une formule de $(\mathcal{L}1\mathcal{O}, \models)$, il nous faut définir la notion d'*atome* qui s'appuie elle même sur la notion de *terme*.

Un terme est une expression syntaxique définie par la grammaire suivante :

$$\langle \text{terme} \rangle ::= \langle \text{variable} \rangle \mid \langle \text{constante} \rangle \mid \langle \text{symbole de fonction} \rangle \underbrace{(\langle \text{terme} \rangle, \ldots, \langle \text{terme} \rangle)}_{n \text{ termes}}$$

où n correspond à l'arité du symbole de fonction.

Remarque. *Un terme* n'est pas *une formule.*

Soit a et b des constantes, x et y des variables, f un symbole de fonction unaire (d'arité 1) et g un symbole de fonction binaire (d'arité 2). Ces expressions sont donc des exemples de termes :

$$a \qquad x \qquad f(x) \qquad g(a,x) \qquad g(f(x), g(f(a), b))$$

Un atome est un expression syntaxique définie par la grammaire suivante :

$$\langle \text{atome} \rangle ::= \langle \text{prédicat} \rangle (\underbrace{\langle \text{terme} \rangle, \ldots, \langle \text{terme} \rangle}_{n \text{ termes}})$$

où n correspond à l'arité du prédicat.

Soit P un prédicat unaire, Q un prédicat binaires et a, b, x, y, f et g analogiquement introduits à ci-dessus. Ces expressions sont donc des exemples d'atomes :

$$P(a) \qquad Q(x,b) \qquad P(f(x)) \qquad Q(g(a,x), f(b)) \qquad P(g(f(x), g(f(a), b)))$$

Une formule de la logique du premier ordre est une expression syntaxique de la forme :

$$\begin{aligned}\langle \text{formule} \rangle ::= {} & \langle \text{connecteur d'arité 0} \rangle \mid \langle \text{atome} \rangle \mid \neg(\langle \text{formule} \rangle) \mid \\ & (\langle \text{formule} \rangle \ \langle \text{connecteur binaire} \rangle \ \langle \text{formule} \rangle) \mid \langle \text{quantificateur} \rangle \ \langle \text{variable} \rangle \ (\langle \text{formule} \rangle)\end{aligned}$$

Les connecteurs sont les mêmes qu'en logique propositionnelle et les deux *quantificateurs* sont :

— Le *quantificateur universel* $\forall$ (« pour tout ») : la formule $\forall x\, \varphi$ signifie que φ est vraie quelle que soit la valeur attribuée à x.

— Le *quantificateur existentiel* $\exists$ (« il existe ») : la formule $\exists x\, \varphi$ signifie qu'il existe au moins une valeur de x pour laquelle φ est vraie.

Variables libres et variables liées.

Dans une formule $\forall x\, \varphi$ ou $\exists x\, \varphi$, on dit que φ est la *portée* du quantificateur. Une variable est dite *liée* si elle apparaît dans la portée d'un quantificateur qui la gouverne ; sinon, elle est dite *libre*. Une formule ne contenant aucune variable libre est appelée *formule close*.

Par exemple, dans la formule $\forall x\ P(x) \to Q(x,y)$, la variable x est liée (par le quantificateur $\forall$) tandis que y est libre. Cette distinction est importante : la vérité d'une formule close est indépendante de tout contexte, alors que celle d'une formule ouverte dépend des valeurs attribuées à ses variables libres.

Remarque. *Les quantificateurs $\forall$ et $\exists$ ne sont pas indépendants : on a $\neg\forall x\ \varphi \equiv \exists x\ \neg\varphi$ et $\neg\exists x\ \varphi \equiv \forall x\ \neg\varphi$. Il suffit donc théoriquement d'un seul des deux.*

Exemple complet.

Considérons la signature suivante :

— Constantes : a, b

— Symbole de fonction : f d'arité 1, g d'arité 2

— Symboles de prédicats : P d'arité 1, Q d'arité 2

On peut alors construire la formule suivante :

$$\forall x \, \big(P(x) \to \exists y \, Q(g(x,\, y),\, f(a))\big)$$

> **Remarque.** *Cette formule se lit : « Pour tout x, si $P(x)$ est vrai, alors il existe un y tel que Q est satisfait par le couple $\big(g(x,y),\, f(a)\big)$ ».*

On peut en détailler la structure :

— x et y sont des *variables* ;

— a est une *constante* (symbole de fonction d'arité 0) ;

— $f(a)$ est un *terme* obtenu en appliquant la fonction f à la constante a ;

— $g(x, y)$ est un *terme* obtenu en appliquant la fonction g aux variables x et y ;

— $P(x)$ et $Q(g(x, y),\, f(a))$ sont des *formules atomiques* (ou atomes), formées en appliquant un symbole de prédicat à des termes ;

— $\forall x$ et $\exists y$ sont des *quantificateurs* liant respectivement x et y ;

— $\to$ est un *connecteur logique* (implication).

Retour sur notre fil conducteur.

Nous disposons maintenant des outils pour formaliser proprement le syllogisme de Socrate. Introduisons la signature suivante :

— Constante : *socrate*

— Symboles de prédicats : *Homme* d'arité 1, *Mortel* d'arité 1

Le raisonnement se traduit alors par la base de connaissances :

$$B = \big\{\, \forall x \, \big(Homme(x) \to Mortel(x)\big), \quad Homme(socrate) \,\big\}$$

et l'on souhaite démontrer :

$$B \models Mortel(socrate)$$

> **Remarque.** *En logique propositionnelle, il aurait fallu créer une proposition distincte pour chaque individu. Ici, une seule formule quantifiée $\forall x$ capture la règle générale, applicable à n'importe quelle constante.*

Une technique courante pour réaliser ce type de preuve automatiquement est la *démonstration par réfutation* : plutôt que de prouver $Mortel(socrate)$ directement, on ajoute sa négation à B et l'on cherche une contradiction. En mettant les formules sous forme clausale (disjonctions de littéraux), on obtient :

$$\begin{aligned} C_1 &= \neg Homme(x) \vee Mortel(x) \\ C_2 &= Homme(socrate) \\ C_3 &= \neg Mortel(socrate) \end{aligned}$$

Pour dériver la contradiction à partir de C_1 et C_2, il faut *résoudre* ces deux clauses, i.e. trouver une substitution qui rende $Homme(x)$ identique à $Homme(socrate)$. C'est exactement ce que réalise l'*unification*, présentée dans la section suivante.

3. L'unification

Unification de termes.

On appelle *unification* le processus visant à répondre à la question « Peut-on *substituer* au terme t la variable x ? ».

La *substitution* est l'application qui, à une formule donnée, associe cette même formule avec toutes les occurrences d'un certain terme remplacé, *substitué*, par une variable.

Par exemple, les deux termes t_1 et t_2 suivants :

$$t_1 = f(X,\, g(a)) \qquad \text{et} \qquad t_2 = f(b,\, g(Y))$$

peuvent être unifiés par la substitution $\sigma = \{X \mapsto b,\ Y \mapsto a\}$.

En effet, $\sigma(t_1) = \sigma(t_2) = f(b,\, g(a))$.

> **Remarque.** *L'unification intervient notamment dans les moteurs d'inférence, les systèmes de réécriture de termes, et les prouveurs logiques.*

Unification de prédicats.

L'unification s'étend naturellement aux atomes. Deux atomes $P(s_1, \ldots, s_n)$ et $P(t_1, \ldots, t_n)$ sont unifiables si et seulement s'ils partagent le même symbole de prédicat P et si leurs arguments respectifs sont unifiables deux à deux.

Ce mécanisme est au cœur de la *règle de résolution* : pour démontrer une formule par réfutation, on cherche à unifier un atome $P(t_1, \ldots, t_n)$ avec sa *négation* $\neg P(s_1, \ldots, s_n)$. Si une substitution σ unifie leurs arguments :

$$\sigma\big(P(t_1, \ldots, t_n)\big) = \sigma\big(P(s_1, \ldots, s_n)\big)$$

alors la résolution produit une *clause vide*, notée $\square$, témoin d'une contradiction, donc d'une preuve par l'absurde.

Clôture du fil conducteur.

Nous pouvons à présent achever la preuve amorcée à la section précédente. La substitution $\sigma = \{x \mapsto socrate\}$ unifie $Homme(x)$ avec $Homme(socrate)$.

La résolution de C_1 et C_2 produit alors $Mortel(socrate)$. En appliquant à nouveau la règle de résolution sur ce résultat avec $C_3 = \neg Mortel(socrate)$, on obtient la clause vide $\square$, attestant d'une contradiction : la négation de ce que l'on voulait prouver est incompatible avec la base de connaissances.

> **Remarque.** *Cet exemple illustre le rôle central de l'unification dans la démonstration automatique : sans elle, la règle de résolution ne pourrait pas* instancier *les variables et ne serait donc d'aucune utilité pratique. La question de l'*efficacité *de cette unification, en fonction des structures de données et des algorithmes utilisés, est précisément l'objet du reste de ce rapport.*

II. Structures de données et algorithmes

1. Structures de données représentant un ensemble de termes

a. AST (*Abstract Syntax Tree*)